

CHƯƠNG 2: LẬP TRÌNH GIAO DIỆN VỚI AWT VÀ SWING

Mục tiêu:

Cung cấp cho sinh viên kỹ năng lập trình giao diện sử dụng bộ công cụ java AWT và SWING. Sinh viên nắm kiến thức cơ bản và vận dụng xây dựng một giao diện của một hệ thống cụ thể.

Nội dung:

- Những nguyên tắc thiết kế giao diện.
- Những thư viện, gói xây dựng giao diện: gồm những lớp(class), những giao tiếp (interface) quản lý sự kiện và những thành phần (components) xây dựng nên giao diện người dùng.
 - Bộ quản lý trình bày (layout managers)
 - Xử lý sự kiện
- AWT: Trong mục này, chúng ta sẽ tìm hiểu và sử dụng các đối tượng cơ bản của lập trình giao diện: Các đối tượng chứa(container) cơ bản (Frame, Panel, Dialog). Các đối tượng thành phần (component) cơ bản (Button, Label, TextField, TextArea). Các sự kiện cơ bản của các đối tượng.
- SWING: Java Swing cung cấp một loạt các thành phần giao diện người dùng (UI components) để xây dựng giao diện đồ họa cho ứng dụng Java

Bài 3: Lập trình giao diện với các thành phần AWT.

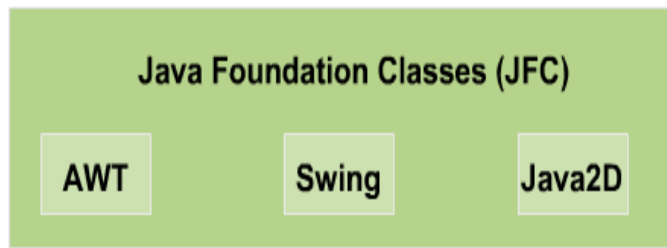
Trong bài này chúng ta sẽ tìm hiểu cơ bản về các thành phần AWT. Các thành phần AWT giúp xây dựng giao diện người dùng trực quan trong ứng dụng Java và là một phần quan trọng của lập trình đồ họa trong Java. Cụ thể chúng ta sẽ tìm hiểu về các thành phần container và các component và các giao diện dùng cho bắt và xử lý sự kiện trong lập trình giao diện với java

2.1. Tổng quan về lập trình giao diện trong java

Giới thiệu JFC (Java Foundation Class):

- Là thư viện lập trình đồ họa phát triển dựa trên thư viện AWT.
- JFC có khả năng tạo nên các giao diện uyển chuyển, linh hoạt hơn AWT.

- Có sẵn trong thư viện chuẩn JDK từ bản 1.2 trở đi.



Hình 2: 1 *Java Foundation Class*

Thư viện awt là bộ thư viện dùng để xây dựng giao diện người dùng cho một chương trình ứng dụng có đầy đủ các thành phần cơ bản như: Label, Button, Checkbox, Radiobutton, Choice, List, Text Field, Text Area, Scrollbar, Menu, Frame...

Giới thiệu Swing: Là một gói nằm trong thư viện JFC, gồm nhiều lớp (classes) và giao diện (interfaces) hỗ trợ mạnh mẽ cho việc lập trình giao diện đồ họa trên JDK.

Java2D chuyên xử lý các thuộc tính giao diện 2 chiều như hình ảnh, text, font, màu sắc,...

2.2. AWT

2.2.1. Các đối tượng container cơ bản

Các đối tượng container được dùng để chứa các đối tượng thành phần khác. Các lớp đối tượng này có một số phương thức chung như sau:

- *add(Object)*: Thêm một đối tượng (kiểu component) vào container.
- *remove(Object)*: Loại bỏ một đối tượng ra khỏi container.
- *removeAll()*: Loại bỏ tất cả các đối tượng mà container đang chứa.
- *getComponent(int)*: Trả về đối tượng thành phần có chỉ số là tham số đầu vào.

Container quản lý các đối tượng chứa trong nó dưới dạng mảng. Chỉ số của các thành phần là số thứ tự khi thành phần đó được thêm vào container.

- *getComponents()*: Trả về mảng tất cả các đối tượng mà container đang chứa.
- *countComponents()*: Trả về số lượng các đối tượng mà container đang chứa.

Frame là một đối tượng có thể dùng một cách độc lập, hoặc được gắn vào một đối tượng khác như một đối tượng component bình thường. Thông thường, Frame được dùng như một cửa sổ của một chương trình độc lập. Các phương thức cơ bản của lớp Frame:

- *Frame()*: Khởi tạo không tham số.
- *Frame(String)*: Khởi tạo với tham số là dòng tiêu đề của frame.

- *setSize(int, int)*: Định kích cỡ của frame, tham số tương ứng là chiều rộng và chiều cao của frame.

- *setVisible(boolean)*: Cho phép frame xuất hiện hay ẩn đi trên màn hình.
- *setTitle(String)/getTitle()*: Truy nhập thuộc tính dòng tiêu đề của frame.
- *setResizable(boolean)*: Thiết lập thuộc tính cho phép thay đổi kích cỡ frame.
- *setIconImage(Image)*: Thiết lập ảnh icon ở góc trên (biểu tượng) của frame.

Panel

Panel cũng là một dạng khung chứa, nhưng khá đơn giản. Panel chỉ dùng để nhóm các đối tượng giao diện với nhau. Thông thường, panel được dùng trong một cửa sổ của Frame hoặc một ứng dụng khác. Các phương thức cơ bản của lớp Panel, ngoài các phương thức chung của container:

- *Panel()*: Khởi tạo không tham số.

Ví dụ: Minh họa việc sử dụng một **Panel** trong một Frame.

```
import java.awt.Button;
import java.awt.Color;
import java.awt.Frame;
import java.awt.Label;
import java.awt.Panel;
import java.awt.TextField;
import javax.swing.JFrame;
import javax.swing.JLabel;
class MyFrame1 extends JFrame {
    Button button1;
    Button button2;
    TextField txt;
    JLabel mylabel = new JLabel("Hello JLabel", JLabel.LEFT);
    MyFrame1(String title) {
        super(title);
        txt = new TextField("abc", 40);
        //txt.setEchoChar('*');
        Panel panel = new Panel();
        Label lblNhan1 = new Label("Nhap Ho Ten");
        panel.setBounds(40, 80, 400, 200);
        panel.setBackground(Color.gray);
        button1 = new Button("Button 1");
```

```

        button1.setBounds(50, 50, 80, 30);
        button1.setBackground(Color.yellow);
        button2 = new Button("Button 2");
        button2.setBounds(50, 100, 80, 30);
        button2.setBackground(Color.green);
        lblNhan1.setBounds(50, 150, 90, 40);
        txt.setBounds(150, 155, 100, 20);
        mylabel.setBounds(150, 50, 120, 30);
        mylabel.setText("Noi dung da thay doi");
        panel.add(txt);
        panel.add(lblNhan1);
        panel.add(button1);
        panel.add(button2);
        panel.add(mylabel);
        panel.setLayout(null);
        this.add(panel);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // this.setSize(400, 400);
        this.setBounds(20, 20, 500, 300);
        this.setLayout(null);
        this.setVisible(true);
    }
}

public class PanelExample1 {
    public static void main(String args[]) {
        new MyFrame1("Vi du 1 AWT trong java");
        // MyFrame1 f1= new MyFrame1("Vi du 1 AWT trong java");
        // System.out.println(f1.txt.getText());
    }
}

```

Dialog

Dialog là một đối tượng cửa sổ con của một cửa sổ chương trình chính. Do vậy, Dialog chỉ được sử dụng kèm với một Frame. Có hai dạng Dialog:

- Modal: Khi hiện của sổ dialog, các cửa sổ khác của chương trình sẽ bị khoá lại, không thao tác được, chỉ thao tác được trên cửa sổ dialog.

- **Non-modal**: Không khoá các cửa sổ khác. Khi dialog xuất hiện, người dùng vẫn có thể chuyển sang thao tác trên các cửa sổ khác, nếu cần.

Các phương thức cơ bản của lớp **Dialog**:

- *Dialog(Frame, boolean)*: Khởi tạo dialog, tham số thứ nhất là frame chứa dialog, tham số thứ hai xác định dialog có là modal hay không.
- *Dialog(Frame, String, boolean)*: Khởi tạo dialog, thêm tham số thứ hai là dòng tiêu đề của dialog.
- *setVisible(boolean)*: Thiết lập trạng thái hiển thị hoặc ẩn dialog trên màn hình.
- *setSize(int, int)*: Định kích cỡ cho dialog, các tham số tương ứng là chiều rộng và chiều cao của dialog.
- *setTitle(String)/getTitle()*: Truy nhập thuộc tính dòng tiêu đề của dialog.
- *setResizable(boolean)*: Thiết lập thuộc tính cho phép thay đổi kích cỡ của dialog.
- *setLayout(Layout)*: Thiết lập chế độ hiển thị các đối tượng chứa trong dialog.

Ví dụ: Minh họa sử dụng đối tượng **Dialog**

```
import java.awt.*;

public class DialogDemo{

    public static void main(String[] args) { // Khai báo và khởi tạo frame có tiêu đề

        Frame myFrame = new Frame("Frame has a dialog!");
        myFrame.setSize(300,150); // Định kích cỡ frame

        // Khai báo và khởi tạo dialog
        Dialog myDialog = new Dialog(myFrame, "An empty dialog!", true);
        myDialog.setSize(300,150); // Định kích cỡ dialog
        myDialog.setVisible(true); // Hiển thị dialog
    }
}
```

2.2.2. Các đối tượng component cơ bản

Label

Label (nhãn) là một đối tượng để hiển thị văn bản tĩnh, những văn bản mà người dùng không thể thay đổi trực tiếp được. Các phương thức cơ bản của Label:

- *Label()*: Khởi tạo một nhãn rỗng.
- *Label(String)*: Khởi tạo một nhãn với nội dung văn bản là tham số đầu vào.

- *Label(String, int)*: Khởi tạo một nhãn có nội dung sẵn, tham số thứ hai xác định cách căn lề của nhãn so với khung chứa, bao gồm {Label.CENTER, Label.LEFT, Label.RIGHT}.

- *setText(String)/getText()*: Truy nhập nội dung văn bản của nhãn.
- *setAlignment(int)/getAlignment()*: Truy nhập thuộc tính căn lề của nhãn.
- *setFont(Font)*: Định dạng phông chữ của nhãn.

```
package vidu.chuong2; // sử dụng nhãn trong một frame.
import java.awt.*;
public class LabelDemo{
    public static void main(String[] args) {// Khai báo và khởi tạo frame có tiêu đề
        Frame myFrame = new Frame("Frame has a label!");
        myFrame.setSize(300,150); // Định kích cỡ frame
        Label myLabel = new Label();// Khai báo và khởi tạo label
        myLabel.setText("This is a label!");//Gán nội dung văn bản
        myLabel.setAlignment(Label.CENTER);// Căn lề giữa
        myFrame.add(myLabel); // Gắn label vào frame
        myFrame.setVisible(true); // Hiển thị frame}
    }
```

TextField và TextArea

Đây là hai đối tượng dùng để biểu diễn văn bản và người dùng có thể thay đổi nội dung văn bản chứa trong chúng. Điểm khác biệt là **TextField** chỉ cho phép một dòng văn bản, trong khi **TextArea** cho phép chứa nhiều dòng văn bản. Các phương thức chung của hai lớp này:

- *setText(String)/getText()*: Truy nhập thuộc tính nội dung văn bản chứa trong ô.
- *getSelectedText()*: Trả về chuỗi văn bản được bôi đen (đánh dấu chọn) trong ô.
- *getSelectedStart()*: Trả về vị trí kí tự đầu trong vùng được đánh dấu chọn (tính từ 0).
- *getSelectedEnd()*: Trả về vị trí kí tự cuối trong vùng được đánh dấu chọn (tính từ 0).
- *selectAll()*: Đánh dấu chọn toàn văn bản.
- *setEditable(boolean)*: Xác định vùng văn bản có thể edit được hay không.

Các phương thức khác của lớp TextField:

- *TextField()*: Khởi tạo một ô văn bản rỗng.
- *TextField(int)*: Khởi tạo một ô văn bản rỗng, độ rộng xác định bởi tham số vào.

- *TextField(String)*: Khởi tạo một ô văn bản có nội dung xác định bởi tham số đầu vào.

- *TextField(String, int)*: Khởi tạo với nội dung có sẵn, độ rộng xác định.

- *setEchoChar(char)/getEchoChar()*: Truy nhập thuộc tính là kí tự thay thế văn bản trong ô. Thuộc tính này được dùng khi ta cần che dấu thông tin văn bản, ví dụ, ô gõ mật khẩu của chương trình.

- *getColumns()*: Trả về độ rộng của ô văn bản.

Các phương thức khác của lớp TextArea:

- *TextArea()*: Khởi tạo một vùng văn bản rỗng.

- *TextArea(int, int)*: Khởi tạo một vùng văn bản rỗng, kích cỡ (số dòng, số cột) xác định bởi tham số vào.

- *TextArea(String)*: Khởi tạo một vùng văn bản có nội dung xác định bởi tham số đầu vào.

- *TextArea(String, int, int)*: Khởi tạo vùng văn bản với nội dung có sẵn, độ rộng xác định.

- *appendText(String)*: Thêm một đoạn văn bản vào cuối đoạn văn bản trong vùng.

- *insertText(String, int)*: Chèn một đoạn văn bản vào vị trí xác định (tham số thứ hai) của vùng văn bản.

- *replaceText(String, int, int)*: Thay thế một đoạn văn bản trong vùng, đánh dấu bằng vị trí bắt đầu và vị trí kết thúc (tham số thứ hai và thứ ba), bằng một đoạn văn bản mới (tham số thứ nhất).

- *getRows()/getColumns()*: Trả về số dòng/cột của vùng văn bản.

```
package vidu.chuong2; // đặt các đối tượng ô văn bản và vùng văn bản vào m
ột
import java.awt.*;
public class TextDemo{
public static void main(String[] args) {// Khai báo và khởi tạo frame có t
iêu đề
Frame myFrame = new Frame("Frame has some texts!");
myFrame.setSize(300,150); // Định kích cỡ frame
// Khai báo và khởi tạo textField
TextField myTextField = new TextField("A text field!");
myFrame.add(myTextField); // Gắn vào frame
```

```
// Khai báo và khởi tạo textArea
TextArea myTextArea = new TextArea(5, 40);
String str="The TextField's columns is: "+myTextField.getColumns();
str += "The TextArea's size is: " + myTextArea.getRows() + "*" + myTextAre
a.getColumns();
myTextArea.setText(str); // Thiết lập nội dung
myFrame.add(myTextArea); // Gắn vào frame
myFrame.setVisible(true); // Hiển thị frame}
}
```

Button

Button là đối tượng nút lệnh, dùng để thực hiện một nhiệm vụ xác định. Các phương thức cơ bản của nút nhấn:

- *Button(String)*: Khởi tạo nút nhấn với tên xác định trên nút.
- *setLabel(String)/getLabel()*: Truy nhập tên của nút nhấn.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.JFrame;
public class DemoButton extends JFrame implements ActionListener
{
    Button helloButton,blueButton,whiteButton,b;
    DemoButton(String title)
    {super(title);
    helloButton=new Button("Hello");
    blueButton=new Button("Blue");
    whiteButton=new Button("White");
    b=new Button("OK");
    this.setLayout(new FlowLayout());
    this.add(helloButton);
    this.add(blueButton);
    this.add(whiteButton);
    this.add(b);
    helloButton.addActionListener(this);
    blueButton.addActionListener(this);
    whiteButton.addActionListener(this);
    b.addActionListener(new ActionListener() {
        @Override
```

```

        public void actionPerformed(ActionEvent e) {
            //To change body of generated methods, choose Tools | Template
s.
            System.out.println("nhan vao ok");
        }
    });
    this.addWindowListener(new WindowClosing());
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setBounds(50,50,300,200);
    this.setVisible(true);

}
public static void main(String args[])
{new DemoButton("DEMO BUTTON");
}
    public void actionPerformed(ActionEvent e)
    { if(e.getSource()==helloButton)
        {System.out.println("Nhấn Helo Button");
        javax.swing.JOptionPane.showMessageDialog(this,"Xin chào");
        }
    if(e.getSource()==blueButton)
        {System.out.println("Nhấn Blue Button");
        }
    if(e.getSource()==whiteButton)
        {System.out.println("Nhấn White Button");
        }
    }
}
}

```

Chạy và quan sát chương trình khi nhấn vào các nút tương ứng.

2.2.3. Các sự kiện cơ bản của đối tượng

Mỗi đối tượng component có một số sự kiện xác định, phát sinh từ chính đối tượng đó. Java cung cấp một số lớp sự kiện cơ bản nằm trong thư gói java.awt.event:

```
import java.awt.event.*;
```

Các lớp sự kiện cơ bản của các đối tượng bao gồm:

- **ActionEvent**: Xuất hiện khi một nút bị click vào, một danh sách (list) được chọn, một menu được chọn.
- **ComponentEvent**: Xuất hiện khi component bị thay đổi kích cỡ, vị trí, trạng thái.
- **FocusEvent**: Xuất hiện khi một component có hoặc mất focus.
- **ItemEvent**: Xuất hiện khi một menu item được chọn hoặc bỏ, khi checkbox hoặc list item được click vào.
- **WindowEvent**: Xuất hiện khi một cửa sổ được mở ra, kích hoạt, đóng lại hoặc thoát ra.
- **TextEvent**: Xuất hiện khi giá trị văn bản của các đối tượng TextField và TextArea bị thay đổi.
- **MouseEvent**: Xuất hiện khi chuột được click, di chuyển, nhấn xuống và thả ra.

- **KeyEvent**: Xuất hiện khi có đầu vào từ bàn phím.

Các giao tiếp được cài đặt để xử lý các sự kiện trên:

- **ActionListener**.
- **ComponentListener**
- **FocusListener**
- **ItemListener**
- **WindowListener**
- **TextListener**
- **MouseListener** và **MouseMotionListener**
- **KeyListener**

Khi cài đặt các giao tiếp này, cần cài đặt lại phương thức xử lý sự kiện:

```
public void actionPerformed(<Đối tượng lớp sự kiện>){
... // Cài đặt lại mã lệnh
}
```

Để xác định sự kiện phát sinh từ component nào, ta dùng phương thức getSource():

<Kiểu component> <Đối tượng sự kiện>.getSource();

Cài đặt một ứng dụng hoàn chỉnh, bao gồm:

- Hai nhãn tiêu đề cho hai ô văn bản.
- Hai ô văn bản, để nhập số liệu vào.

- Bốn nút nhấn tương ứng để thực hiện các thao tác nhân, chia, cộng, trừ các số liệu nhập từ hai ô văn bản.
- Thêm một nút nhấn, khi click vào sẽ thoát khỏi chương trình.

```
package vidu.chuong2;
import java.awt.*;
import java.awt.event.*;
public class EventDemo extends Frame implements ActionListener{
    Label lbl1, lbl2, lblKq;
    TextField txt1, txt2;
    Button btnCong, btnTru, btnNhan, btnChia, btnThoat;
    public EventDemo(){
        super("Event demo!");
        this.setLayout(new GridLayout(6,2)); //Chế độ hiển thị 6 dòng, 2 cột
        lbl1 = new Label("So thu nhat:"); // Nhãn số thứ nhất
        this.add(lbl1);
        txt1 = new TextField(); // Ô văn bản số thứ nhất
        this.add(txt1);
        lbl2 = new Label("So thu hai:"); // Nhãn số thứ hai
        this.add(lbl2);
        txt2 = new TextField(); // Ô văn bản số thứ hai
        this.add(txt2);
        lblKq = new Label(); // Nhãn kết quả
        this.add(lblKq);
        this.add(new Label());
        btnCong = new Button("Cong"); // Nút cộng
        btnCong.addActionListener(this); // Bắt sự kiện
        this.add(btnCong);
        btnTru = new Button("Tru"); // Nút trừ
        btnTru.addActionListener(this);
        this.add(btnTru);
        btnNhan = new Button("Nhan"); // Nút nhân
        btnNhan.addActionListener(this);
        this.add(btnNhan);
        btnChia = new Button("Chia"); // Nút chia
        btnChia.addActionListener(this);
```

```

this.add(btnChia);
btnThoat = new Button("Thoat"); // Nút thoát
btnThoat.addActionListener(this);
this.add(btnThoat);
// Phương thức bắt sự kiện click vào nút đóng frame
this.addWindowListener(new WindowAdapter(){
    public void windowClosing(WindowEvent e){
        System.exit(0);}
});
}
/* Phương thức xử lí sự kiện nút được nhấn */
public void actionPerformed(ActionEvent ae){
    float x = Float.parseFloat(txt1.getText());
    float y = Float.parseFloat(txt2.getText());
    float kq = 0;
    if(ae.getSource() == btnCong) // Cộng hai số
        kq = x + y;
    if(ae.getSource() == btnTru) // Trừ hai số
        kq = x - y;
    if(ae.getSource() == btnNhan) // Nhân hai số
        kq = x*y;
    if(ae.getSource() == btnChia)&&(y != 0) // Chia hai số
        kq = x/y;
    if(ae.getSource() == btnThoat) // Thoát khỏi chương trình
        System.exit(0);
    // Thay đổi nội dung kết quả
    lblKq.setText("Ket qua la: " + String.valueOf(kq)); }
public static void main(String[] args) { // Khai báo đối tượng demo
    EventDemo myFrame = new EventDemo();
    myFrame.setSize(300,150); // Định kích cỡ frame
    myFrame.setVisible(true); // Hiển thị frame
}
Checkbox("Red", cbxg, true); // Nút red
cbxRed.addItemListener(this); // Bắt sự kiện
this.add(cbxRed);

```

```

cbxBlue = new Checkbox("Blue", cbxg, false); // Nút blue
cbxBlue.addItemListener(this); // Bắt sự kiện
this.add(cbxBlue);
cbxGreen = new Checkbox("Green", cbxg, false); // Nút green
cbxGreen.addItemListener(this); // Bắt sự kiện
this.add(cbxGreen);
// Phương thức bắt sự kiện click vào nút đóng frame
this.addWindowListener(new WindowAdapter(){
public void windowClosing(WindowEvent e){
System.exit(0);
}
});
}
/* Phương thức xử lý sự kiện thay đổi trạng thái nút */
public void itemStateChanged(ItemEvent ie){
if(ie.getStateChanged() == ItemEvent.SELECTED){
String item = (String)ie.getItem();
if(item.equals("Red")) // Đổi màu red
this.setBackground(Color.red);
if(item.equals("Blue")) // Đổi màu blue
this.setBackground(Color.blue);
if(item.equals("Green")) // Đổi màu green
this.setBackground(Color.green);
this.repaint(); // Vẽ lại màu nền
}
}
public static void main(String[] args) { // Khai báo đối tượng demo
RadioDemo myFrame = new RadioDemo();
myFrame.setSize(300,150); // Định kích cỡ frame
myFrame.setVisible(true); // Hiển thị frame
}
}

```

2.2.4. Lựa chọn(Choice)

Choice là đối tượng menu sổ xuống, hiển thị một danh sách các item và cho phép người dùng chọn một trong số các item đó (tương tự đối tượng dropdown list của window). Các phương thức cơ bản của lớp Choice:

- *Choice()*: Khởi tạo đối tượng choice.
- *addItem(String)*: Thêm một item vào danh sách lựa chọn.

- *remove(int)*: Xoá item ở vị trí thứ i trong danh sách (bắt đầu là vị trí 0).
- *removeAll()*: Xoá toàn bộ item trong danh sách chọn.
- *select(int)/select(String)*: Chọn một item theo số thứ tự hoặc theo tên.
- *getSelectedIndex()*: Trả về chỉ số của item được chọn.
- *getSelectedItem()*: Trả về tên của item được chọn.
- *getItem(int)*: Trả về tên của item tương ứng với số thứ tự đưa vào.

Xử lý sự kiện thay đổi trạng thái nút chọn:

- Kiểu sự kiện: *ItemEvent*
- Cài đặt giao tiếp: *ItemListener*
- Phương thức xử lý: *itemStateChanged(ItemEvent)*

```
package vidu.chuong2;// Thay đổi màu nền theo màu được chọn sử dụng choice
import java.awt.*;
import java.awt.event.*;
public class ChoiceDemo extends Frame implements ItemListener{
    Choice myChoice;
    public ChoiceDemo(){
        super("Choice demo!");
        myChoice = new Choice(); // Khởi tạo
        myChoice.addItem("Red"); // Thêm item red
        myChoice.addItem("Blue"); // Thêm item blue
        myChoice.addItem("Green"); // Thêm item green
        myChoice.addItemListener(this); // Bắt sự kiện
        this.add(myChoice); // Gắn vào frame
        // Phương thức bắt sự kiện click vào nút đóng frame
        this.addWindowListener(new WindowAdapter(){
            public void windowClosing(WindowEvent e){
                System.exit(0);
            }
        });
    }
    /* Phương thức xử lý sự kiện thay đổi trạng thái item */
    public void itemStateChanged(ItemEvent ie){
        if(ie.getStateChanged() == ItemEvent.SELECTED){
            String item = (String)ie.getItem();
            if(item.equals("Red")) // Đổi màu red
```

```

this.setBackground(Color.red);
if(item.equals("Blue")) // Đổi màu blue
this.setBackground(Color.blue);
if(item.equals("Green")) // Đổi màu green
this.setBackground(Color.green);
this.repaint(); // Vẽ lại màu nền}
}

public static void main(String[] args) { // Khai báo đối tượng demo
ChoiceDemo myFrame = new ChoiceDemo();
myFrame.setSize(300,150); // Định kích cỡ frame
myFrame.setVisible(true); // Hiển thị frame
}
}

```

2.2.5. Danh sách

List là một danh sách hoạt động tương tự đối tượng choice. Tuy nhiên, list cho phép người dùng có thể chọn một hoặc nhiều item một lúc. Các phương thức cơ bản:

- *List()*: Khởi tạo một danh sách rỗng, mỗi lần chỉ được chọn một item.
- *List(int)*: Tương tự, nhưng có qui định số dòng được nhìn thấy.
- *List(int, boolean)*: Khởi tạo một danh sách có số dòng được nhìn thấy xác định, chế độ cho phép chọn một hay nhiều item xác định bởi tham số thứ hai.
- *add(String)*: Thêm một item vào danh sách.
- *add(String, int)*: Chèn một item vào vị trí xác định trong danh sách. Nếu chỉ số chèn vượt ra khỏi phạm vi danh sách, item sẽ được thêm vào cuối.
- *replaceItem(String, int)*: Thay thế một item ở vị trí xác định (tham số thứ hai) trong danh sách bằng một item mới (tham số thứ nhất).
- *remove(int)*: Xoá item ở vị trí xác định trong danh sách.
- *removeAll()*: Xoá toàn bộ item hiện có của danh sách.
- *getSeletedIndex()*: Trả về index của item được chọn (danh sách đơn chọn).
- *getSelectedItem()*: Trả về item được chọn (danh sách đơn chọn).
- *getSelectedIndexes()*: Trả về chỉ số các item được chọn (danh sách đa chọn).
- *getSelectedItems()*: Trả về các item được chọn (danh sách đa chọn).

Xử lý sự kiện khi thay đổi item được chọn:

- Kiểu sự kiện: **ItemEvent**

- Cài đặt giao tiếp: **ItemListener**
- Phương thức xử lí: *itemStateChange(ItemEvent)*;

Ví dụ: sử dụng đối tượng list với khả năng đa chọn. Mỗi khi thay đổi item được chọn, một thông báo các màu được chọn sẽ hiện ra.

```
package vidu.chuong2;
import java.awt.*;
import java.awt.event.*;
public class ListDemo extends Frame implements ItemListener{
    List myList;
    Label lbl;
    public ListDemo(){
        super("List demo!");
        // Khởi tạo list đa chọn, chỉ nhìn được một dòng
        myList = new List(1, true);
        myList.setSize(300,150);
        // Thêm các item là các loại màu sắc
        myList.add("White");
        myList.add("Red");
        myList.add("Orange");
        myList.add("Green");
        myList.add("Yellow");
        myList.add("Blue");
        myList.add("Black");
        myList.addItemListener(this); // Bắt sự kiện
        this.setLayout(new FlowLayout());
        this.add(myList); // Gắn vào frame
        lbl = new Label(); // Khởi tạo nhãn
        this.add(lbl); // Gắn vào frame
        // Phương thức bắt sự kiện click vào nút đóng frame
        this.addWindowListener(new WindowAdapter(){
            public void windowClosing(WindowEvent e){
                System.exit(0);
            }
        });
    }
    /* Phương thức xử lí sự kiện thay đổi trạng thái item */
}
```

```

public void itemStateChange(ItemEvent ie){
    if((ie.getStateChange() == ItemEvent.SELECTED)|| (ie.getStateChange() == I
    temEvent.DESELECTED)){
        String kq = "Cac mau duoc chon:";
        String[] items = myList.getSelectedItems();
        for(int i=0; i<items.length; i++)
            kq += items[i] + ", ";
        lbl.setText(kq);
    }
}

public static void main(String[] args) {
    ListDemo myFrame = new ListDemo();// Khai báo đối tượng demo
    myFrame.setSize(300,150); // Định kích cỡ frame
    myFrame.setVisible(true); // Hiển thị frame
}
}

```

2.2.6. Trình đơn(Memu)

Trình đơn (menu) được dùng trên các thanh công cụ của các cửa sổ hoặc là popup menu xuất hiện khi ta click chuột phải vào một đối tượng. Java cung cấp một số lớp trình đơn:

- **Menubar**: Thanh trình đơn
- **Menu**: Trình đơn đồ xuống
- **PopupMenu**: Trình đơn xuất hiện khi click chuột phải.
- **MenuItem**: Các mục chọn của trình đơn.

Menubar

Menubar là thanh công cụ dùng để chứa các trình đơn menu. Các phương thức cơ bản của lớp Menubar:

- *Menubar()*: Khởi tạo một thanh công cụ cho trình đơn
- *add(Menu)*: Thêm một trình đơn mune lên thanh trình đơn menubar.

Để đặt một menubar của một frame, ta gọi phương thức của frame:

<Đối tượng frame>.setMenuBar(<Đối tượng menubar>);

Menu và PopupMenu

Trình đơn menu là đối tượng sẽ xổ xuống khi click chuột lên đối tượng hiển thị của menu. Menu còn được gọi là menu con của một thanh trình đơn. Các phương thức cơ bản của lớp Menu:

- *Menu(String)*: Khởi tạo một menu, với tên xác định.

- *add(MenuItem)*: Thêm một item vào menu
- *add(Menu)*: Thêm một menu con vào menu đã có, dùng khi muốn tạo menu có nhiều mức.
- *addSeparator()*: Thêm một đường phân vùng vào menu (để nhóm các item với nhau).

Xử lý sự kiện của lớp **Menu**:

- Kiểu sự kiện: **ActionEvent**
- Giao tiếp cài đặt: **ActionListener**
- Phương thức xử lý: *actionPerformed(ActionEvent)*;

MenuItem

MenuItem là đối tượng item trong các trình đơn menu. Mỗi item, khi được click vào sẽ có tác dụng như một nút lệnh. Các phương thức cơ bản của lớp MenuItem:

- *MenuItem(String)*: Khởi tạo một item.
- *CheckboxMenuItem(String)*: Khởi tạo một item có mục chọn như checkbox.
- *getState()*: Trả về trạng thái của item. Chỉ dùng cho item có mục chọn.
- *enable()*: Cho phép item hoạt động (là chế độ mặc định).
- *disable()*: Không cho phép item hoạt động (làm mờ item đi).

Xử lý sự kiện của lớp **MenuItem**:

- Kiểu sự kiện: **ActionEvent**
- Giao tiếp cài đặt: **ActionListener**
- Phương thức xử lý: *actionPerformed(ActionEvent)*;

Ví dụ: minh hoạ việc sử dụng các loại menu:

- Tạo một menubar của frame
- Trên menubar, tạo một menu “File”. Khi click vào sẽ xổ xuống một menu với các item: New, Open, Save, Save As, Exit.
- Khi bấm chuột phải vào frame, sẽ xổ ra một menu popup gồm các item: Cut, Copy, Paste.
- Khi click chuột vào item nào trên các menu, một nhãn trong frame sẽ hiển thị tên của item vừa được chọn.
- Chương trình kết thúc khi click vào item Exit.

```
package vidu.chuong2;
import java.awt.*;
import java.awt.event.*;
```

```

public class MenuDemo extends Frame
implements ActionListener, MouseListener{
Menubar myBar;
Menu myMenu;
PopupMenu myPopup;
Label lbl;
public MenuDemo(){
super("Menu demo!");
myBar = new Menubar(); // Thanh trình đơn
this.setMenuBar(myBar); // Thiết lập menubar của frame
myMenu = new Menu("File"); // menu File
myBar.add(myMenu); // Gắn menu vào thanh trình đơn
myMenu.addActionListener(this); // Bắt sự kiện
myMenu.add(new MenuItem("New"));
myMenu.add(new MenuItem("Open"));
myMenu.addSeparator(); // Thêm dấu nhóm item
myMenu.add(new MenuItem("Save"));
myMenu.add(new MenuItem("Save As"));
myMenu.addSeparator();
myMenu.add(new MenuItem("Exit"));
myPopup = new PopupMenu("Options"); // Menu popup
myPopup.addActionListener(this); // Bắt sự kiện
myPopup.add(new MenuItem("Cut"));
myPopup.add(new MenuItem("Copy"));
myPopup.add(new MenuItem("Paste"));
lbl = new Label(); // Khởi tạo nhãn
this.add(lbl); // Gắn vào frame
// Phương thức bắt sự kiện click vào nút đóng frame
this.addWindowListener(new WindowAdapter(){
public void windowClosing(WindowEvent e){
System.exit(0);
}
});
}
/* Phương thức xử lý sự kiện */
public void actionPerformed(ActionEvent ae){

```

```

if(ae.getActionCommand().equals("Exit")){
    System.exit(0);
}
lbl.setText(ae.getActionCommand());
}
public void mouseEntered(MouseEvent me){} // Không xử lí
public void mouseExited(MouseEvent me){} // Không xử lí
public void mouseReleased(MouseEvent me){} // Không xử lí
public void mousePressed(MouseEvent me){} // Không xử lí
public void mouseClicked(MouseEvent me){
    myPopup.show(this, me.getX(), me.getY()); // Hiện menu popup
}
public static void main(String[] args) {
    // Khai báo đối tượng demo
    MenuDemo myFrame = new MenuDemo();
    myFrame.setSize(300,150); // Định kích cỡ frame
    myFrame.setVisible(true); // Hiển thị frame
}
}

```

Bài 4: Lập trình giao diện với các thành phần AWT và SWING(tiếp theo)

Tiếp nối phần lập trình giao diện, nội dung trong bài này chúng ta tìm hiểu về khái niệm quản lý trình bày các thành phần với Layout Manager. Cụ thể chúng ta sẽ tìm hiểu một số kiểu trình bày cơ bản cũng như cách dùng chúng trong trình bày bố cục của giao diện, phần tiếp theo sẽ là các giao diện của AWT trong bất sự kiện cho con trỏ chuột và bàn phím. Tiếp theo chúng ta sẽ tìm hiểu thư viện SWING được phát triển từ AWT với các thành phần đồ họa cao cấp được sử dụng trong lập trình giao diện.

2.2.7. Layout Manager

Nội dung phần này sẽ tập trung trình bày các kỹ thuật trình bày các đối tượng giao diện (component) trên frame theo các ý đồ thiết kế khác nhau bằng cách dùng bộ quản lý trình bày (Layout Manager). Bao gồm các kỹ thuật sau:

1) Trình bày Flow Layout

Đề sắp xếp các đối tượng trên một hướng theo dòng. Nếu đối tượng mới thêm không đủ chỗ (chiều rộng) thì nó sẽ tự động thêm vào đầu dòng mới. Các phương thức:

- *FlowLayout()*: Khởi tạo đối tượng trình bày.
- *FlowLayout(int)*: Khởi tạo đối tượng trình bày với cách căn lề xác định.

- *FlowLayout(int, int, int)*: Khởi tạo với ba tham số: Thứ nhất là cách căn lề, thứ hai là khoảng cách giữa hai dòng (chiều cao), thứ ba là khoảng cách giữa hai đối tượng (chiều ngang).

Tham số căn lề có thể nhận một trong ba giá trị:

- *FlowLayout.LEFT*: Căn lề trái, là giá trị mặc định.
- *FlowLayout.CENTER*: Căn lề giữa.
- *FlowLayout.RIGHT*: Căn lề phải.

package vidu.chuong2; // cách trình bày flow layout: Tạo ra một dãy 10 nút nhấn và gắn vào một frame theo kiểu flow layout.

```
import java.awt.*;
public class FlowLayoutDemo{
    public static void main(String[] args) { // Khai báo và khởi tạo frame có tiêu đề
        Frame myFrame = new Frame("Frame has somes buttons!");
        myFrame.setSize(300,150); // Định kích cỡ frame
        myFrame.setLayout(new FlowLayout( ));// Thiết lập cách trình bày
        // Khai báo và khởi tạo button
        for(int i=0; i<10; i++)
            myFrame.add(new Button("Click"+i));// Gắn vào frame
        myFrame.setVisible(true); // Hiển thị frame
    }
}
```

2) Trình bày Grid Layout

Cách trình bày Grid Layout sẽ sắp xếp các đối tượng theo dạng bảng, được xác định số hàng và số cột. Phương thức cơ bản:

- *GridLayout(int, int)*: Khởi tạo một đối tượng trình bày. Hai tham số đầu vào lần lượt là số hàng và số cột của grid trình bày.

- *GridLayout(int, int, int, int)*: Khởi tạo một đối tượng trình bày, hai tham số đầu xác định số hàng và số cột trình bày. Hai tham số sau xác định khoảng cách giữa các dòng và các cột của bảng.

Lưu ý:

Khi số lượng đối tượng được chèn nhiều hơn vào frame, ta muốn chương trình tự tính số hàng, hoặc tự tính số cột hiển thị, thì ta để tham số tương ứng là 0.

Ví dụ: `setLayout(new GridLayout(3,0));` // cố định số hàng là 3, số cột tùy thuộc vào số đối tượng trong frame.

`setLayout(new GridLayout(0,2));` // cố định số cột là 2, số dòng mềm dẻo theo số các đối tượng trong frame.

Ví dụ: minh họa cách trình bày grid layout: Tạo ra một dãy 10 nút nhấn và gắn vào một frame theo kiểu grid layout.

```
package vidu.chuong6;
import java.awt.*;
public class GridLayoutDemo{
public static void main(String[] args) {
// Khai báo và khởi tạo frame có tiêu đề
Frame myFrame = new Frame("Frame has some buttons!");
myFrame.setSize(300,150); // Định kích cỡ frame
myFrame.setLayout(new GridLayout(0,2)); // Thiết lập cách trình bày
// Khai báo và khởi tạo button
for(int i=0; i<10; i++)
myFrame.add(new Button("Click"+i)); // Gắn vào frame
myFrame.setVisible(true); // Hiển thị frame
}
```

3) Trình bày Border Layout

Cách hiển thị Border Layout sẽ chia frame thành 5 vùng cố định và tự động kéo giãn các vùng sao cho chiếm hết bề mặt của frame:

- West: Vùng phía tây, tức là phía lề bên trái.
- East: Vùng phía đông, tức là phía lề bên phải.
- North: Vùng phía bắc, tức là phía lề trên.
- South: Vùng phía nam, tức là phía lề dưới.
- Center: Vùng trung tâm, ở chính giữa frame.

Phương thức cơ bản của lớp BorderLayout:

- `BorderLayout()`: Khởi tạo một đối tượng trình bày theo cách border.

Khi một frame được trình bày theo cách border, ta có thể dùng phương thức sau để gắn các đối tượng vào các vùng của frame:

`<Đối tượng frame>.add(<Vùng border>, <Đối tượng component>);`

Ví dụ: `myFrame.add("Center", new Button("Click"));`

sẽ gắn vào vùng trung tâm của myFrame một nút nhấn có tên là "Click".

Lưu ý:

- Cách trình bày border luôn chia frame thành 5 vùng xác định.
- Nếu gắn nhiều đối tượng vào cùng một vùng, chỉ có đối tượng gắn sau là nhìn thấy được.
- Nếu muốn trong một vùng chứa được nhiều đối tượng, ta có thể gắn vào mỗi vùng một Panel. Sau đó trong panel, ta chọn cách trình bày riêng cho panel và gắn các đối tượng vào panel.

```
//minh hoạ cách trình bày border:
package vidu.chuong2;
import java.awt.*;
public class BorderLayoutDemo{
    public static void main(String[] args) { // Khai báo và khởi tạo frame có tiêu đề
        Frame myFrame = new Frame("Frame has somes buttons!");
        myFrame.setSize(300,150); // Định kích cỡ frame
        myFrame.setLayout(new BorderLayout()); // Định cách trình bày
        // Khai báo và khởi tạo button
        myFrame.add("West", new Button("West")); // Gắn vào vùng west
        myFrame.add("East", new Button("East")); // Gắn vào vùng east
        myFrame.add("North", new Button("North")); // Gắn vào vùng north
        myFrame.add("South", new Button("South")); // Gắn vào vùng south
        // Gắn vào vùng center
        myFrame.add("Center", new Button("Center"));
        myFrame.setVisible(true); // Hiển thị frame
    }
}
```

4) Trình bày GridBag Layout

Cách trình bày GridBag Layout cũng trình bày các đối tượng tương tự như Grid Layout: Các đối tượng sẽ được định vị theo vị trí các ô (cell) của một khung lưới (grid). Tuy nhiên, GridBag cho phép ta định kích thước của đối tượng sẽ chiếm bao nhiêu ô và sẽ được đặt ở vị trí nào trong khung lưới. Các phương thức cơ bản:

- *GridBagLayout()*: Khởi tạo một đối tượng trình bày theo cách gridbag.
- *setConstraints(Component, GridBagConstraints)*: Đặt vị trí và kích thước của đối tượng component theo các ràng buộc trong gridbagConstraints.

GridBagConstraints

Đây là lớp chứa các ràng buộc cho các đối tượng được trình bày theo cách GridBag. Các phương thức và thuộc tính cơ bản của lớp GridBagConstraints:

- *GridBagConstraints()*: Khởi tạo một đối tượng ràng buộc của GridBag.
- *gridx/gridy*: Vị trí của cell mà ta muốn đặt đối tượng vào (theo chiều X và chiều Y).
- *gridwidth/gridheight*: Kích thước (vùng trình bày) của đối tượng (Theo chiều rộng và chiều cao).
- *fill*: Xác định cách đặt đối tượng, theo 4 cách:
 - *GridBagConstraints.NONE*: Đối tượng không thay đổi kích thước theo các cell nó chiếm.
 - *GridBagConstraints.VERTICAL*: Đối tượng có chiều cao kín vùng nó chiếm
 - *GridBagConstraints.HORIZONTAL*: Đối tượng có chiều rộng kín vùng nó chiếm
 - *GridBagConstraints.BOTH*: Đối tượng có chiều cao và chiều rộng phủ kín vùng nó chiếm.
- *ipadx/ipady*: Định đơn vị tăng giảm kích thước của đối tượng khi khung chứa bị thay đổi kích thước (theo chiều X và chiều Y).
- *insets*: Xác định khoảng cách giữa các cell theo bốn hướng: Trên, dưới, trái, phải.
- *anchor*: Xác định vị trí neo đối tượng khi kích thước khung chứa thay đổi.
Bao gồm: NORTH, NORTHEAST, NORTHWEST, EAST, SOUTH, SOUTHEAST, SOUTHWEST.
- *weightx/weighty*: Định khoảng cách lớn ra tương đối giữa các đối tượng với nhau.

//minh hoạ cách trình bày gridbag: Tạo ra ba nút nhấn trong frame, mỗi nút có một số ràng buộc khác nhau về kích thước và vị trí.

```
import java.awt.*;  
  
public class GridBagLayoutDemo{  
    public static void main(String[] args) {  
        // Khai báo và khởi tạo frame có tiêu đề  
        Frame myFrame = new Frame("Frame has somes buttons!");  
        myFrame.setSize(300,150); // Định kích cỡ frame  
        GridBagLayout layout = new GridBagLayout();
```

```

myFrame.setLayout(layout); // Định cách trình bày
// Khai báo đối tượng ràng buộc
GridBagConstraints cts = new GridBagConstraints();
cts.fill = GridBagConstraints.BOTH;
// Button1: vị trí (1,1), kích thước (1,1)
Button btn1 = new Button("Click1");
cts.gridx = 1;
cts.gridy = 1;
cts.gridheight = 1;
cts.gridwidth = 1;
layout.setConstraints(btn1, cts); // Định ràng buộc
myFrame.add(btn1); // Gắn vào frame
// Button2: vị trí (2,2), kích thước (1,1)
Button btn2 = new Button("Click2");
cts.gridx = 2;
cts.gridy = 2;
cts.gridheight = 1;
cts.gridwidth = 1;
layout.setConstraints(btn2, cts); // Định ràng buộc
myFrame.add(btn2); // Gắn vào frame
// Button3: vị trí (3,3), kích thước (1,1)
Button btn3 = new Button("Click3");
cts.gridx = 3;
cts.gridy = 3;
cts.gridheight = 1;
cts.gridwidth = 1;
layout.setConstraints(btn3, cts); // Định ràng buộc
myFrame.add(btn3); // Gắn vào frame
myFrame.setVisible(true); // Hiển thị frame
}
}

```

5) Trình bày Null Layout

Cách trình bày Null Layout sẽ trình bày các đối tượng không theo một quy tắc nào. Tất cả đều do người dùng tự định vị và thiết lập kích thước cho mỗi đối tượng.

- Định vị đối tượng bằng phương thức *setLocation()*:

<Đối tượng>.setLocation(Point);

- Định kích thước đối tượng bằng phương thức setSize():

<Đối tượng>.setSize(int, int);

• Ngoài ra, có thể vừa định vị, vừa định kích thước cho đối tượng thông qua phương thức:

<Đối tượng>.setBounds(int, int, int, int);

Trong đó, hai tham số đầu định vị đối tượng, hai tham số sau định kích thước đối tượng.

Ví dụ: minh họa cách trình bày tự do Null layout: tạo ra hai nút nhấn và gắn vào frame theo hai cách khác nhau.

```
import java.awt.*;

public class NullLayoutDemo{
    public static void main(String[] args) {
        // Khai báo và khởi tạo frame có tiêu đề
        Frame myFrame = new Frame("Frame has somes buttons!");
        myFrame.setSize(300,150); // Định kích cỡ frame
        myFrame.setLayout(null); // Định cách trình bày
        // Button1: vị trí (10,30), kích thước (100,40)
        Button btn1 = new Button("Click1");
        btn1.setSize(100, 40);
        btn1.setLocation(new Point(10, 30));
        myFrame.add(btn1); // Gắn vào frame
        // Button2: vị trí (70,120), kích thước (50,20)
        Button btn2 = new Button("Click2");
        btn2.setBounds(70, 120, 50, 20);
        myFrame.add(btn2); // Gắn vào frame
        myFrame.setVisible(true); // Hiển thị frame
    }
}
```

2.2.8. Xử lý sự kiện

1) Mô hình xử lý sự kiện (Event-Handling Model)

Những sự kiện được phát sinh khi người dùng tương tác với giao diện chương trình (GUI). Những tương tác thường gặp như: di chuyển, nhấn chuột, nhấn một nút nhấn, chọn một MenuItem trong hệ thống thực đơn, nhập dữ liệu trong một ô văn bản, đóng cửa sổ ứng dụng, ... Khi có một tương tác xảy ra thì một sự kiện được gửi đến

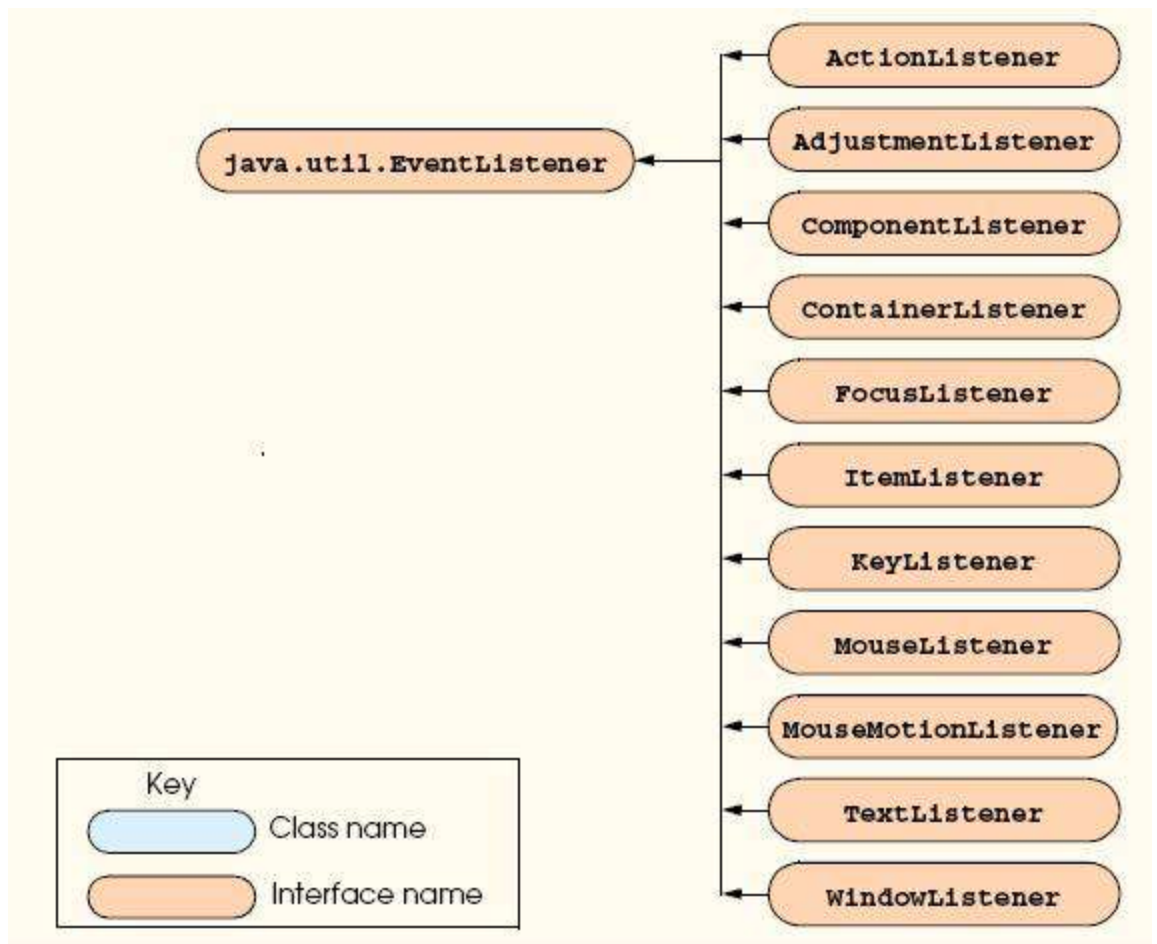
chương trình. Thông tin về sự kiện thường được lưu trữ trong một đối tượng dẫn xuất từ lớp `AWTEvent`. Những kiểu sự kiện trong gói `java.awt.event` có thể dùng cho cả những component AWT và JFC. Đối với thư viện JFC thì có thêm những kiểu sự kiện mới trong gói `java.swing.event`.

Có 3 yếu tố quan trọng trong mô hình xử lý sự kiện:

- Nguồn phát sinh sự kiện (event source): là thành phần của giao diện mà người dùng tác động.
- Sự kiện (event object): Tóm tắt thông tin về xử kiện xảy ra, bao gồm tham chiếu đến nguồn gốc phát sinh sự kiện và thông tin sự kiện sẽ gọi đến cho bộ lắng nghe xử lý.
- Bộ lắng nghe sự kiện (event listener): là một đối tượng của một lớp hiện thực một hay nhiều interface của gói `java.awt.event` hay `java.swing.event` (đối với những component trong thư viện JFC). Khi được thông báo, bộ lắng nghe nhận sự kiện và xử lý.

Nguồn phát sinh sự kiện phải cung cấp những phương thức để đăng ký hoặc hủy bỏ một bộ lắng nghe. Nguồn phát sinh sự kiện luôn phải gắn với một bộ lắng nghe, và nó sẽ thông báo với bộ lắng nghe đó khi có sự kiện phát sinh đó.

2) Những interfaces lắng nghe của gói `java.awt.event`



Hình 2: 2 *interfaces* lắng nghe sự kiện

3) Xử lý sự kiện chuột

Các phương thức của interface **MouseListener**:

- *public void mousePressed(MouseEvent event)*: được gọi khi một nút chuột được nhấn và con trỏ chuột ở trên component.
- *public void mouseClicked(MouseEvent event)*: được gọi khi một nút chuột được nhấn và nhả trên component mà không di chuyển chuột.
- *public void mouseReleased(MouseEvent event)*: được gọi khi một nút chuột nhả sau khi kéo rê.
- *public void mouseEntered(MouseEvent event)*: được gọi khi con trỏ chuột vào trong đường biên của một component.
- *public void mouseExited(MouseEvent event)*: được gọi khi con trỏ chuột ra khỏi đường biên của một component.

Các phương thức của interface **MouseMotionListener**:

- *public void mouseDragged(MouseEvent even)*: phương thức này được gọi khi người dùng nhấn một nút chuột và kéo trên một component.

- *public void mouseMoved(MouseEvent event)*: phương thức này được gọi khi di chuyển chuột trên component.

Mỗi phương thức xử lý sự kiện chuột có một tham số **MouseEvent** chứa thông tin về sự kiện chuột phát sinh chẳng hạn như: tọa độ x, y nơi sự kiện chuột xảy ra. Những phương thức tương ứng trong các interfaces sẽ tự động được gọi khi chuột tương tác với một component.

Để biết được người dùng đã nhấn nút chuột nào, chúng ta dùng những phương thức, những hằng số của lớp **InputEvent** (là lớp cha của lớp **MouseEvent**).

4) Xử lý sự kiện bàn phím

Để xử lý sự kiện bàn phím java hỗ trợ một bộ lắng nghe sự kiện đó là **interface KeyListener**. Một sự kiện bàn phím được phát sinh khi người dùng nhấn và nhả một phím trên bàn phím. Một lớp hiện thực **KeyListener** phải cài đặt các phương thức *keyPressed*, *keyReleased* và *keyTyped*. Mỗi phương thức này có một tham số là một đối tượng kiểu **KeyEvent**. **KeyEvent** là lớp con của lớp **InputEvent**.

Các phương thức của **interface KeyListener**

- Phương thức *keyPressed* được gọi khi một phím bất kỳ được nhấn.

Phương thức *keyTyped* được gọi thực hiện khi người dùng nhấn một phím không phải “phím hành động” (như phím mũi tên, phím Home, End, Page Up, Page Down, các phím chức năng như: Num Lock, Print Screen, Scroll Lock, Caps Lock, Pause).

- Phương thức *keyReleased* được gọi thực hiện khi nhả phím nhấn sau khi sự kiện *keyPressed* hoặc *keyTyped*.

2.3. SWING

2.3.1. Giới thiệu.

Swing là thư viện lập trình mở rộng của Java. Nó mở rộng các đối tượng giao diện đồ họa cơ bản của Java. Swing còn được gọi là thư viện JFC (Java Foundation Class). Khi muốn sử dụng các đối tượng đồ họa của thư viện này, ta phải khai báo chỉ thị:

```
import javax.swing.*;
```

- So sánh giữa Swing và AWT:

AWT	SWING
Là toolkit, không có một qui tắc chuẩn và người dùng khi sử dụng có thể nhúng từng phần riêng lẻ	Là framework, có một chuẩn lập trình riêng (MVC), khi người dùng sử dụng các thành phần thì phải tuân theo chuẩn này

Pluggable-Look-And-Feel Hỗ trợ giao diện tương tác nhưng giữa các platform sẽ hiển thị khác nhau	Look-And-Feel Hỗ trợ giao diện tương tác và giữa các platform có thể hiển thị giống nhau hoặc khác nhau (tùy chọn)
HeavyWeight component Các component của AWT là các Peer-Component nên sẽ tốn tài nguyên hơn do với mỗi Platform nó phải có bộ peer component riêng, bộ render của AWT là native-code (C/C++)	Lightweight component Các component của Swing sử dụng chung bộ component chuẩn của Swing nên đỡ tốn bộ nhớ hơn, bộ render sử dụng là thuần Java
Số lượng component và hiệu ứng còn hạn chế	Số lượng component và hiệu ứng phong phú

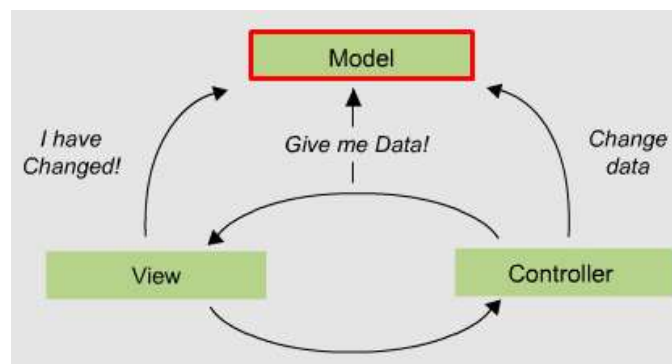
Bảng 2: 1 So sánh SWING và AWT

Vậy có một câu hỏi đặt ra là Swing có phải là sự thay thế cho AWT hay không?

Câu trả lời là không, bởi vì thật ra Swing kế thừa từ AWT, nó có đầy đủ tính năng của AWT và chỉ bổ sung thêm các thành phần để cải thiện hoạt động cho các thành phần của AWT

- **Kiến trúc MVC của swing**

Mô hình tổng quát của MVC như sau:



Hình 2: 3 Kiến trúc MVC của swing

MODEL: Lưu trữ dữ liệu, trạng thái của các component. Ví dụ chúng ta có một Button thì các dữ liệu của Button như chiều dài, chiều rộng, label,... của button sẽ được lưu trữ trong model

VIEW: Xử lý hiển thị giao diện của ứng dụng với các dữ liệu đã có trong Model

CONTROLLER: Xử lý các sự kiện khi người dùng thao tác trên View

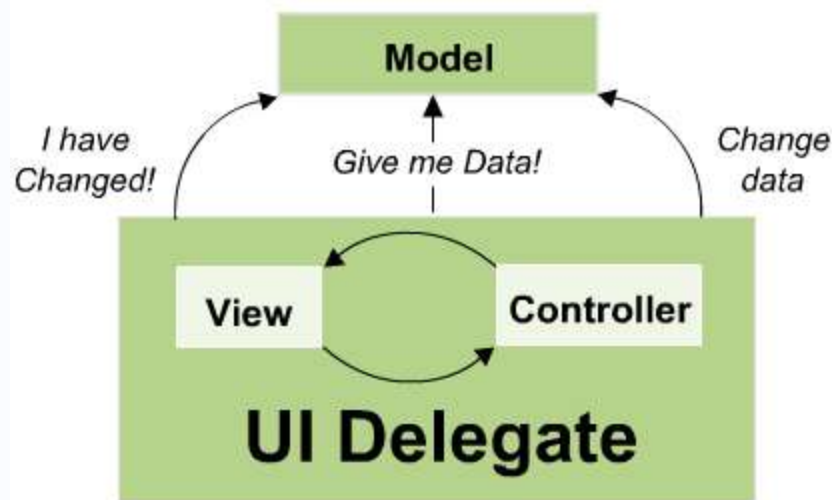
Quy trình xử lý của mô hình MVC như sau:

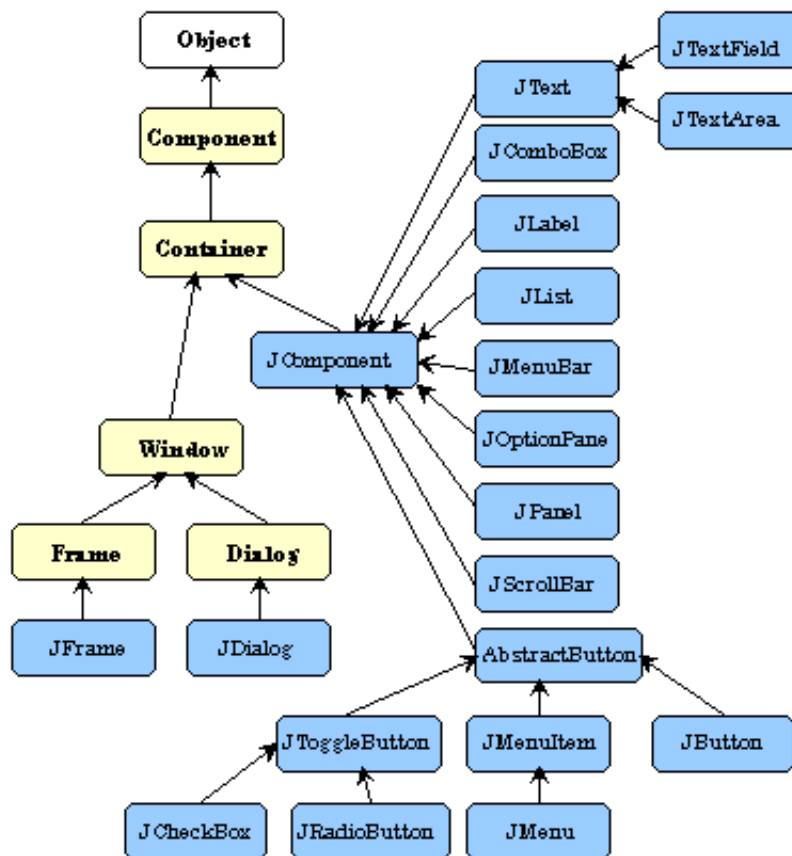
MODEL lưu trữ các dữ liệu được thiết lập ban đầu của các component trong chương trình. Khi đó VIEW sẽ lấy những dữ liệu này và render lên phía giao diện.

Nếu người dùng thao tác trên một component nào đó của tầng VIEW thì sẽ phát sinh sự kiện và việc phải làm gì khi sự kiện phát sinh sẽ giao cho CONTROLLER.

Trong quá trình xử lý sự kiện nếu có cập nhật lại dữ liệu thì CONTROLLER sẽ gửi dữ liệu thay đổi cho MODEL và MODEL tiến hành cập nhật lại. Lúc này VIEW cũng cập nhật theo cho đúng với dữ liệu mới.

Đối với MVC trong SWING có chút cải tiến, khi nó gom thành phần VIEW và CONTROLLER lại với nhau, gọi là Separable model architecture. Lý do của sự phân chia này vì thật khó để viết một Controller khi mà không có mối quan hệ khăng khít với View để lấy các thuộc tính của nó.





Hình 2: 4 Các lớp cơ bản trong swing:

2.3.2. Mở rộng các đối tượng component

JFC mở rộng các đối tượng cơ bản của java thành các lớp tương ứng, ngoài trừ việc có thêm chữ “J” ở đầu mỗi tên lớp:

Button – JButton, Label -JLabel, TextField – JTextField, TextArea - JTextArea

Checkbox- JCheckbox, List - JList, Menu - JMenu

Các lớp mở rộng này có đầy đủ các phương thức của các đối tượng lớp cơ bản của thư viện java.awt. Ngoài ra, chúng bổ sung một số phương thức tạo hiệu ứng giao diện.

Ví dụ: minh họa việc sử dụng đối tượng JButton. Đối tượng JButton được mở rộng thêm một số tính năng sau:

- *JButton(String, Icon)*: Khởi tạo một nút nhấn với một tên nhãn và một ảnh nền. Ảnh nền có kiểu icon (tham số thứ hai).
- *setMnemonic(char)*: Định phím tắt cho nút lệnh. Khi người dùng nhấn “Ctrl+phím tắt” thì nút lệnh cũng thực thi tương tự như khi ta click chuột vào nút lệnh.
- *setBorder(new MatteBorder(int, int, int, int, Icon))*: Thiết lập khung nền cho nút với các tham số: Khoảng cách từ chữ đến biên (độ rộng biên) theo các chiều trên dưới, trái phải, cuối cùng là ảnh nền cho nút.

- `setBorder(new LineBorder(int))`: Thiết lập viền cho nút dạng hình chữ nhật, tham số xác định màu cho viền của nút. Ngoài ra, tham số của phương thức này còn có thể là các lớp **SoftBevelBorder**, **EtchedBorder** và **TitleBorder**.

- `setToolTipText(String)`: Thiết lập dòng tooltip cho đối tượng. Dòng này sẽ hiển ra khi ta di chuột lên đối tượng trên cửa sổ.

```
import javax.swing.*;

public class JButtonDemo extends JFrame{

    public static void main(String[] args) { // Khai báo và khởi tạo frame có tiêu đề

        JFrame myFrame = new JFrame("Frame has somes buttons!");
        myFrame.setSize(300,150); // Định kích cỡ frame
        // Giả sử ta có file ảnh myImage trong cùng thư mục
        Icon myIcon = new ImageIcon("myImage.gif");
        // Button1: có nền là ảnh
        JButton btn1 = new JButton("Back Image", myIcon);
        // Gán tooltip cho nút
        btn1.setToolTipText("Button's background is an image");
        myFrame.getContentPane().add(btn1); // Gắn vào frame
        // Button2: có biên là ảnh
        JButton btn2 = new JButton("Border Image");
        // Gán tooltip cho nút
        btn1.setToolTipText("Button's border is an image");
        btn2.setBorder(new MatteBorder(10,10,10,10, myIcon));
        myFrame.getContentPane().add(btn2); // Gắn vào frame
        myFrame.setVisible(true); // Hiển thị frame
    }
}
```

Trong chương trình này, có dòng lệnh gắn các đối tượng vào frame bằng cách `getContentPane()`. Đây là phương thức mở rộng cho các đối tượng khung chứa container.

- Các đối tượng cơ bản khác: (sinh viên tự tìm hiểu các phương thức thông thường của các đối tượng trong docs java).



Hình 2: 5 Các đối tượng cơ bản khác của java SWING

2.3.3. Mở rộng các đối tượng container

Tương tự như các đối tượng component, các đối tượng container cũng được mở rộng trong JFC thành các lớp có tên tương ứng và thêm kí tự “J” ở đầu: **JFrame**, **JPanel**, **JDialog**, **Jmenu**...

Ví dụ: minh họa việc sử dụng các đối tượng mở rộng của khung chứa Frame thành JFrame. Khung chứa JFrame có nhiều tầng trình diễn khác nhau, các tầng là trong suốt và chồng khít lên nhau, khiến cho ta vẫn có cảm giác các đối tượng được trình bày trên cùng một mặt phẳng như khung chứa Frame của thư viện chuẩn AWT.

Một số tầng hay sử dụng của lớp JFrame (theo thứ tự từ trong ra ngoài):

- **ContentPane**: Là tầng thường dùng nhất, tầng này dùng để chứa các đối tượng component cơ bản như button, label, text, list...
- **MenuBarPane**: Tầng dành để chứa các loại menu của frame như MenuBar, PopupMenu.
- **GlassPane**: Tầng ngoài cùng, thường dùng để chứa các tooltip của các đối tượng trong tầng Content. Khi ta set tooltipText cho một đối tượng, tooltip đó sẽ tự động được add vào tầng Glass.

Để truy nhập vào một tầng bất kì, ta dùng phương thức có tên:

get + <Tên của tầng>();

Ví dụ: `JFrame myFrame = new JFrame("My JFrame");`

`myFrame.getContentPane().add("Center", new JButton("Test"));`

sẽ gắn một nút nhấn có nhãn Test vào tầng Content của khung chứa myFrame.

Ví dụ: minh họa việc gắn các đối tượng vào các tầng khác nhau:

- Gắn một nút nhấn vào tầng **ContentPane**.
- Gắn một thanh MenuBar có chứa một menu File vào tầng MenuBarPane.

```
import javax.swing.*;
```

```

import java.awt.event.*;

public class JFrameDemo extends JFrame implements ActionListener{
    private JMenuBar myBar;
    private JMenu myMenu;
    public JFrameDemo(){
        super("JFrame demo");
        JButton btn = new JButton();
        // Gắn nút nhấn vào tầng ContentPane
        this.getContentPane().add("Center", btn);
        myBar = new JMenuBar();
        myMenu = new JMenu("File");
        myMenu.add(new JMenuItem("Open"));
        myMenu.add(new JMenuItem("New"));
        myMenu.add(new JSeparator());
        myMenu.add(new JMenuItem("Save"));
        myMenu.add(new JMenuItem("Save As"));
        myMenu.add(new JSeparator());
        myMenu.add(new JMenuItem("Exit"));
        myMenu.addActionListener(this);
        myBar.add(myMenu);
        this.setJMenuBar().add(myBar); // Gắn menubar vào tầng MenubarPane
        // Phương thức bắt sự kiện click vào nút đóng frame
        this.addWindowListener(new WindowAdapter(){
            public void windowClosing(WindowEvent e){
                System.exit(0);
            }
        });
    }
    /* Phương thức xử lí sự kiện */
    public void actionPerformed(ActionEvent ae){
        if(ae.getActionCommand().equals("Exit")){
            System.exit(0);
        }
    }
    public static void main(String[] args) {
        // Khai báo và khởi tạo frame có tiêu đề
        JFrameDemo myFrame = new JFrameDemo();
    }
}

```

```
myFrame.setSize(300,150); // Định kích cỡ frame
myFrame.setVisible(true); // Hiển thị frame
}
}
```

2.3.4. Các đối tượng nâng cao của swing

JformatTextField

JEditorPane

JTextPane

DialogBox

JOptionPane

JDialog

File Dialog Box gồm: **JFileChooser**, **JToolBar**,...



Hình 2: 6 Các đối tượng nâng cao của SWING

Câu hỏi ôn tập.

1. Tác dụng của việc dùng các layout manager.
2. So sánh awt và swing
3. Các đối tượng giao diện, các phương thức cơ bản của các đối tượng đó.
4. Các phương thức xử lý sự kiện.
5. Trình bày cách tạo giao diện sử dụng Java SWING và AWT.
6. Lấy các ví dụ minh họa cho các đối tượng giao diện đã học.

BÀI TẬP THỰC HÀNH

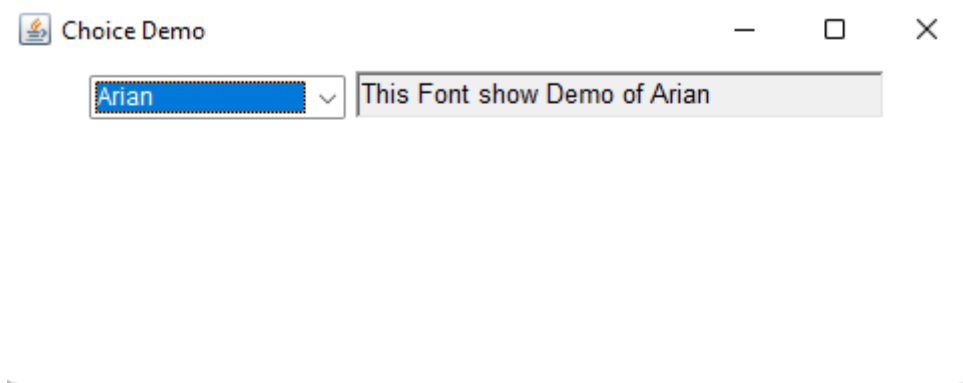
Bài 1.

Lập trình bắt sự kiện cho JFrame có giao diện như sau:

a).



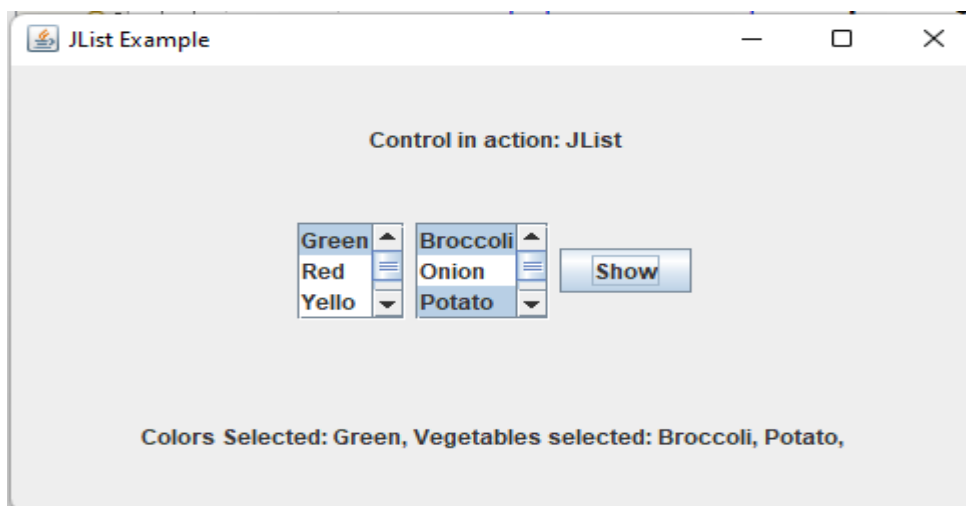
b).



Bài 2.

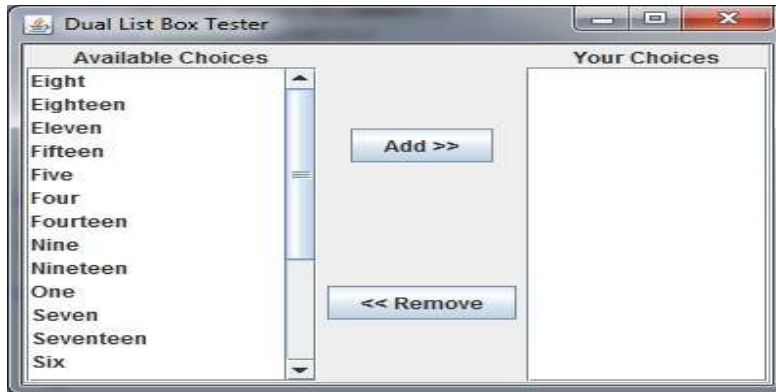
Sử dụng JList xây dựng chương trình có giao diện như sau:

Yêu cầu: khi người dùng nhấn vào button Show các thông tin được lấy từ JList sẽ được hiển thị bằng JLabel như trên:



Bài 3.

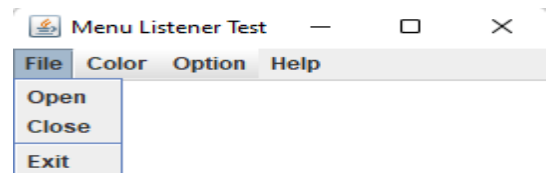
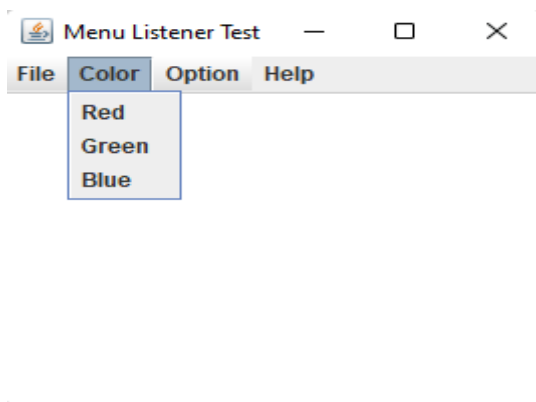
Xây dựng giao diện có giao diện như sau: Bắt sự kiện với hai button “Add>>” và “<<Remove”:



Bài 4.

Xây dựng giao diện và xử lý sự kiện với Menu trong java với giao diện như sau:

-Yêu cầu: Bắt sự kiện với các MenuItem: Red-Green-Bulue để thay đổi màu nền của JFrame, Exit để thoát chương trình (setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE), với các MenuItem khác thực hiện console VD: Item “Open” selected.



Bài 5.

Sử dụng kết hợp các kiểu LayoutManager xây dựng giao diện sau:

